

Lab Task: Library Management System

Design and implement a C++ **Library Management System** that demonstrates core Object-Oriented Programming principles. Your implementation must satisfy the following requirements:

1. Class Structure & Encapsulation

- **Book Class:**
 - **Private Data Members:** `title` (string), `id` (int), `quantity` (int).
 - **Constructor:** Parameterized constructor to initialize members.
 - **Display:** Provide a `virtual void display()` function that prints the title, ID, and quantity.
 - **Friendship:** Grant the `Library` class `friend` access to the `Book` class to allow access to private members.

2. Polymorphism & Functionality

- **Search Functionality:** Implement two overloaded `search()` functions to search by `id` (int) and `title` (string), displaying the book details if found.
- **Operator Overloading:**
 - Overload operators to allow direct increase/decrease of `quantity` by an integer.
 - Overload the `+` operator to combine two `Book` objects *only* when they share the same `id` (adding their quantities together).
- **Ebook Class (Inheritance):**
 - Inherit from the `Book` class.
 - Add a private member `fileSize` (double).
 - Override the `display()` function to show the standard book details plus the `fileSize`.

3. Library Management & Exception Handling

- **Library Class:**
 - Maintain a collection of `Book` objects (e.g., using `std::vector`).
 - **Methods:**
 - `addBook()`: Adds a book to the collection.
 - `displayBooks()`: Lists all books currently in the library.
 - `borrowBook(int id)`: Decreases the quantity of the specific book by 1.
 - `returnBook(int id)`: Increases the quantity of the specific book by 1.
 - **Exception Handling:** Utilize `try-catch` blocks to handle errors. Specifically, throw a `runtime_error` if an attempt is made to borrow a book that is unavailable (quantity 0) or if the `id` provided does not exist.

4. Main Driver Program

In your `main()` function, perform the following sequence:

1. Instantiate the `Library` and add at least two `Book` objects.
2. Display all books.
3. Demonstrate the `search()` functionality by searching for one book by `id` and another by `title`.

4. Inside a `try-catch` block:
 - Successfully borrow a valid book.
 - Return that book.
 - Attempt to borrow a book with an invalid `id` to trigger the exception.
5. Instantiate one `EBook` object and call its `display()` function to demonstrate the overridden behavior.

C++ OOP Programming Tasks

Below are 5 advanced programming tasks designed to reinforce your skills in Inheritance, Polymorphism, Operator Overloading, and Exception Handling.

Task 1: Hospital Management System

Objective: Manage patient records and doctor appointments with specific validation rules.

- **Classes:**
 - `Person` (Base): Name, Age, ID.
 - `Doctor` (Derived): Specialty, Experience.
 - `Patient` (Derived): Diagnosis, AssignedDoctorID.
- **Requirements:**
 - Implement `virtual void displayInfo()` to output class-specific details.
 - Overload `==` to compare two `Doctor` objects based on their ID.
 - Create a `Hospital` class to manage collections.
 - **Exception Handling:** Throw a `runtime_error` in `bookAppointment()` if the doctor ID does not exist or if the doctor is currently "on-leave."

Task 2: Banking Account System

Objective: Develop a robust banking ledger system.

- **Classes:**
 - `Account` (Base): AccountNumber, Balance.
 - `SavingsAccount` (Derived): InterestRate.
 - `CheckingAccount` (Derived): OverdraftLimit.
- **Requirements:**
 - Use `virtual void calculateInterest()` to apply different interest logic for Savings vs. Checking.
 - Overload `+=` for deposits and `-=` for withdrawals.
 - **Exception Handling:** Ensure `-=` throws an exception if the withdrawal amount is negative or exceeds the available balance (or overdraft limit).

Task 3: Employee Payroll System

Objective: Calculate compensation for different employee types.

- **Classes:**
 - `Employee` (Base): Name, ID.

- `FullTimeEmployee` (Derived): Fixed salary.
- `PartTimeEmployee` (Derived): Hourly rate, hours worked.
- **Requirements:**
 - Implement `virtual double calculatePay()` for compensation logic.
 - Overload `*` to allow a `FullTimeEmployee` object to be multiplied by a "performance multiplier" (double) to calculate bonuses.
 - **Exception Handling:** Throw an exception if `hoursWorked` is negative or `performanceRating` is out of the 0.0–2.0 range.

Task 4: Vehicle Rental Management

Objective: Manage a fleet of vehicles with dynamic pricing.

- **Classes:**
 - `Vehicle` (Base): Brand, Model, IsRented.
 - `Car` (Derived): SeatingCapacity.
 - `Bike` (Derived): EngineCC.
- **Requirements:**
 - Implement `virtual double calculateRentalCost(int days)` (Cars cost more per day).
 - Overload `<<` (friend function) to print vehicle status to the console.
 - **Exception Handling:** In the `RentalAgency` class, throw an error if an attempt is made to rent a vehicle already marked as `IsRented = true`.

Task 5: Quiz Game Application

Objective: Build an extensible, interactive quiz engine.

- **Classes:**
 - `Question` (Base): Text, Marks.
 - `MCQ` (Derived): Options array, CorrectOptionIndex.
 - `TrueFalse` (Derived): IsTrue (bool).
- **Requirements:**
 - Use `virtual bool askQuestion()` to manage input based on question type.
 - Overload `++` on a `UserScore` class to increment scores.
 - **Exception Handling:** Throw an `invalid_argument` exception if the user inputs an answer choice outside the valid range (e.g., entering 'E' for a 4-option MCQ).